COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010- Data Structures and Algorithms | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | 9/15/26 | **DATE SUBMITTED:** | 9/15/26 |
| **NAME:** | Vince Andrew Urbano | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. 9.1** **Implementing Trees**

## 1. PROCEDURE OUTPUT

Code:

```cpp
#ifndef tree_h
#define tree_h
#include <iostream>

template <typename T>
class BinaryTree {
    private:
    //key
    T key;
    //left child and right child
    BinaryTree<T>* leftChild;
    BinaryTree<T>* rightChild;

    public:

    //constructor
    BinaryTree<T>(T rootObj);
    //insert children left and right
    void insertLeft(T newNode);
    void insertRight(T newNode);
    // to access the child nodes
    BinaryTree<T>* getLeftChild();
    BinaryTree<T>* getRightChild();

    // get root value & set root node
    T getRootVal();
    void setRootVal(T obj);
    //node checker

    bool isLeaf();
```

```cpp
template <typename T>
BinaryTree<T>::BinaryTree(T rootObj) {
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
}

template <typename T>
T BinaryTree<T>::getRootVal() {
    return key;
}
template <typename T>
void BinaryTree<T>::insertLeft(T newNode) {
    // two types

    // if there currently no child
    if (leftChild == nullptr) {
        leftChild = new BinaryTree<T>(newNode);
    }
    // if we want to insert a certain node
    else {
        //create a new Node
        BinaryTree<T>* newChild = new BinaryTree<T>(newNode);

        //move the existing left child
        newChild->leftChild = leftChild;
```

```
61          //make new node the left child
62          leftChild = newChild;
63
64      }
65  }
66  template <typename T>
67  void BinaryTree<T>::setRootVal(T obj) {
68      key = obj;
69  }
70
71
72
73  template <typename T>
74  BinaryTree<T>* BinaryTree<T>::getLeftChild() {
75      return leftChild;
76  }
77
78  template <typename T>
79  void BinaryTree<T>::insertRight(T newNode) {
80      if (rightChild == nullptr) {
81          rightChild = new BinaryTree<T>(newNode);
82      }else {
83          BinaryTree<T>* newChild = new BinaryTree<T>(newNode);
84          newChild->rightChild = rightChild;
```

```
    }
}

template <typename T>
BinaryTree<T>* BinaryTree<T>::getRightChild() {
    return rightChild;
}
template <typename T>
bool BinaryTree<T>::isLeaf() {
    return leftChild == nullptr && rightChild == nullptr;
}
#endif #ifndef tree_h
```

```
#include "main.h"
#include <iostream>

int main() {

    BinaryTree<char>* root = new BinaryTree<char>('V');
    std::cout <<"The root is: " << root->getRootVal() << std::endl;


    //creating the child of node V
    root->insertLeft('B');
    std::cout<<"The left child is: " << root->getLeftChild()->getRootVal() << std::endl;

    root->insertRight('C');
    std::cout<<"The right child is: " << root->getRightChild()->getRootVal() << std::endl;

    root->getLeftChild()->insertLeft('D');
    std::cout <<"The left child of B is: " << root->getLeftChild()->getLeftChild()->getRootVal() << std::endl;
    root->getLeftChild()->insertRight('E');
    std::cout <<"The right child of B is: " << root->getLeftChild()->getRightChild()->getRootVal() << std::endl;

    root->getRightChild()->insertLeft('F');
    std::cout <<"The left child of C is: " << root->getRightChild()->getLeftChild()->getRootVal() << std::endl;
    root->getRightChild()->insertRight('G');
    std::cout <<"The right child of C is: " << root->getRightChild()->getRightChild()->getRootVal() << std::endl;

    return 0;
}
```

Output:

```
The root is: V
The left child is: B
The right child is: C
The left child of B is: D
The right child of B is: E
The left child of C is: F
The right child of C is: G
```

Code Analysis:
In this activity, we have implemented a binary tree. The header file consists of different child classes of the parent class named BinaryTree.The child classes have different functions, first is the getRootVal. GetRootVal is responsible for displaying the key or the root of the tree that we are creating. Next are the functions for creating the branches of the root. InsertLeft and InsertRight are responsible for adding a value on the left and right side of the root. Next is the setRootVal, it is responsible for setting a value as the root node of the tree. IsLeaf function is responsible for checking if the root has no child.

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.